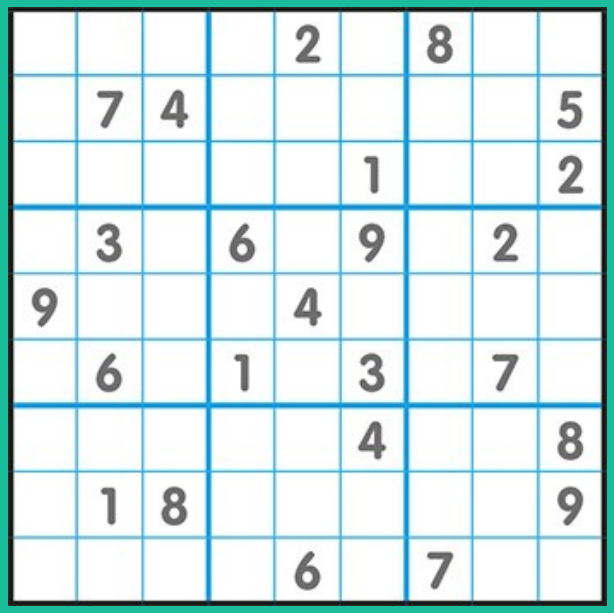
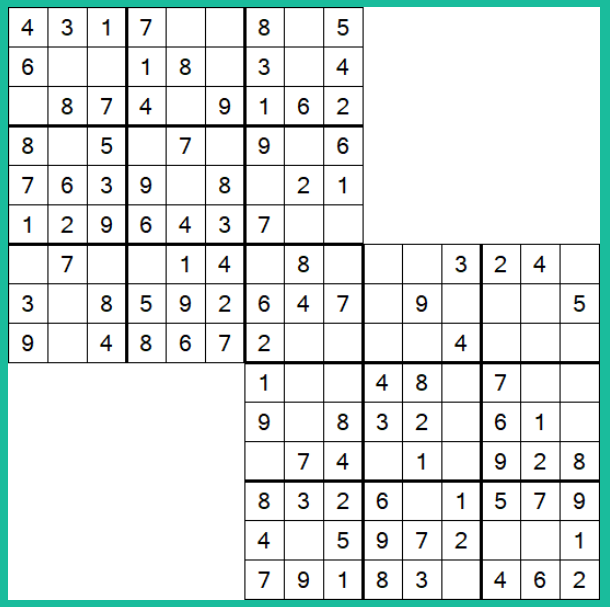
Variantes de Sudoku : résolution optimisée à l'aide de résolveur SAT



https://sudoku.megastar.fr/grille-de-sudoku-niveau-7-a-8/



https://sudoku-puzzles.net/fr/twodoku-simple/

36714
VEILLON Camille
2024

TIPE – Jeux, sports

Comment peut-on adapter un SAT-solver pour optimiser la résolution de variantes de Sudoku ?

I – Logique Propositionnelle

a) Problème et codage SAT

b) Codage du Sudoku vers SAT

II – SAT Solver

a) 1ère idée : Algorithme DPLL

b) 2ème idée : Algorithme CDCL

c) Premiers tests

III – Changements de contraintes

a) Contrainte supplémentaire

b) Optimisation : Preprocessing

c) Contrainte de forme

d) Contrainte de règles

e) Résultats

IV – Conclusion et améliorations

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

e) Résultats

V - Conclusion

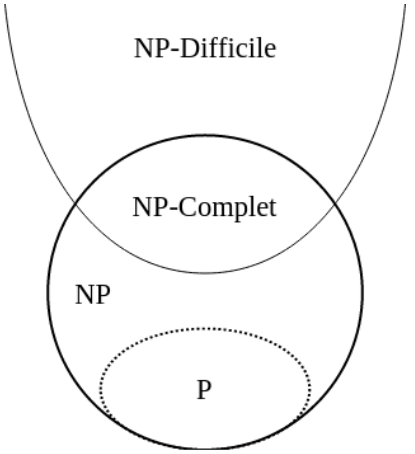
I – a) Problème et codage SAT

Problème SAT :

Donnée : Un formule de la logique propositionnelle avec k variables

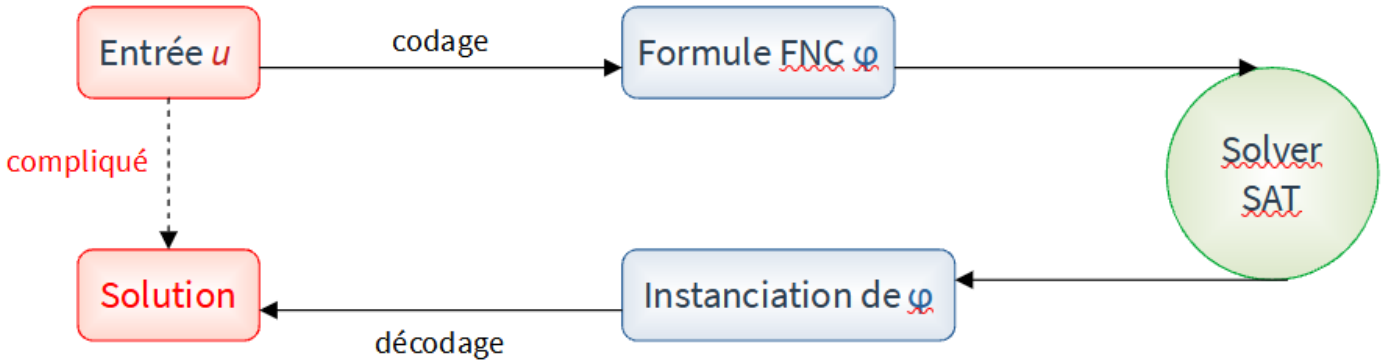
Sortie : Vrai, si la formule est satisfiable, Faux sinon

Sudoku et SAT sont NP-complets (Cook-Levin 1971)



https://www.researchgate.net

Codage SAT :



I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

e) Résultats

V - Conclusion

I – b) Codage Sudoku vers SAT

Pour une grille 4x4 :

Grille remplie : $\bigwedge_{i=1}^{4} \bigwedge_{j=1}^{4} \bigvee_{k=1}^{4} x_{i,j}^{k}$

Unicité sur les lignes $\bigwedge_{k=1}^{4} \bigwedge_{j=1}^{4} \bigwedge_{1<i1<i2<4}^{4} (\neg x_{i1,j}^{k} \vee \neg x_{i2,j}^{k})$

nicité sur les colonnes : $\bigwedge_{k=1}^{4} \bigwedge_{i=1}^{4} \bigwedge_{1<j1<j2<4}^{4} (\neg x_{i,j1}^{k} \vee \neg x_{i,j2}^{k})$

Unicité dans les sous-grilles 2x2 :

$$\bigwedge_{k=1}^{4} \bigwedge_{r=0}^{1} \bigwedge_{s=0}^{1} \bigwedge_{1<i1<i2<2}^{} \bigwedge_{1<j1<j2<2}^{} (\neg x_{2r+i1,2s+j1}^{k} \vee \neg x_{2r+i2,2s+j2}^{k})$$

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

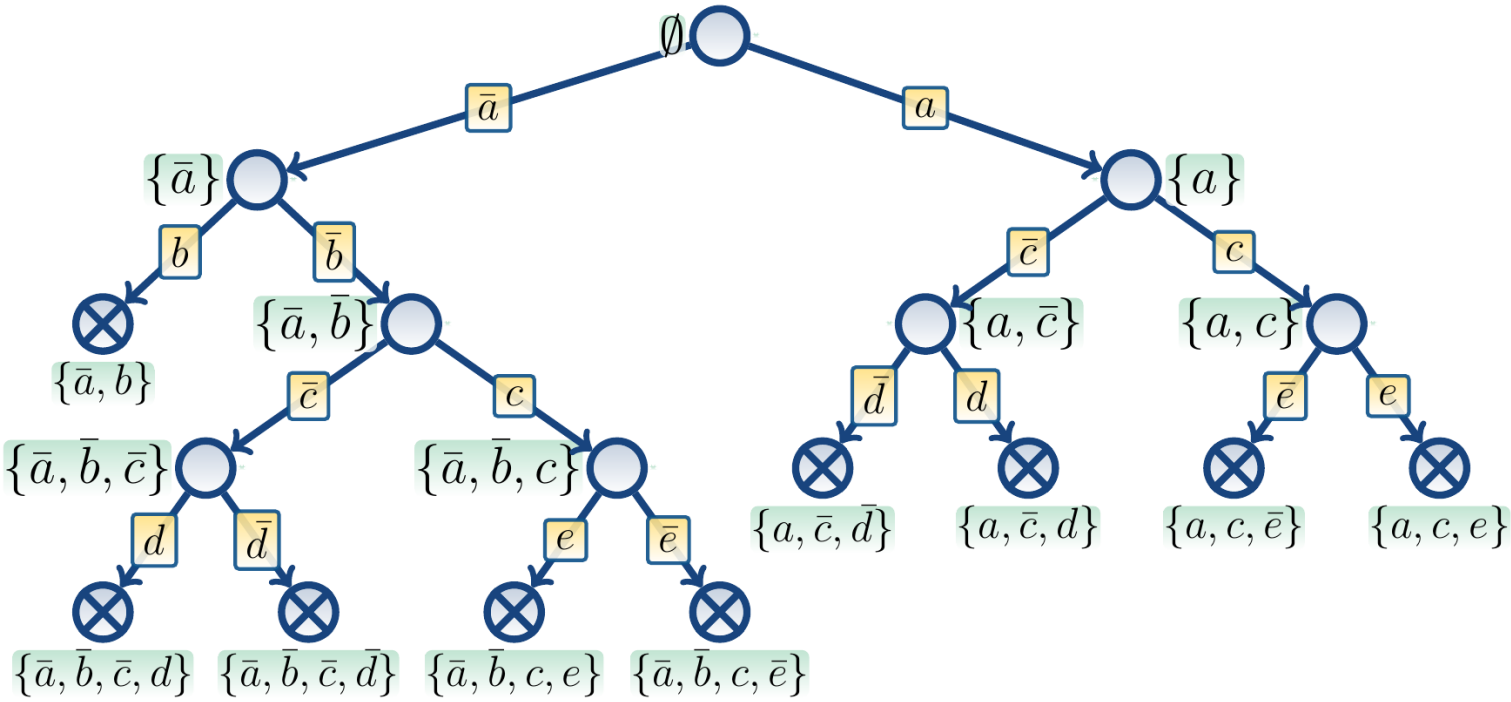
d) Contrainte de règle

e) Résultats

V - Conclusion

II – a) Algorithme DPLL

Algorithme de retour sur trace complet



https://users.aalto.fi/~tjunttil/2020-DP-AUT/notes-sat/dpll.html

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

e) Résultats

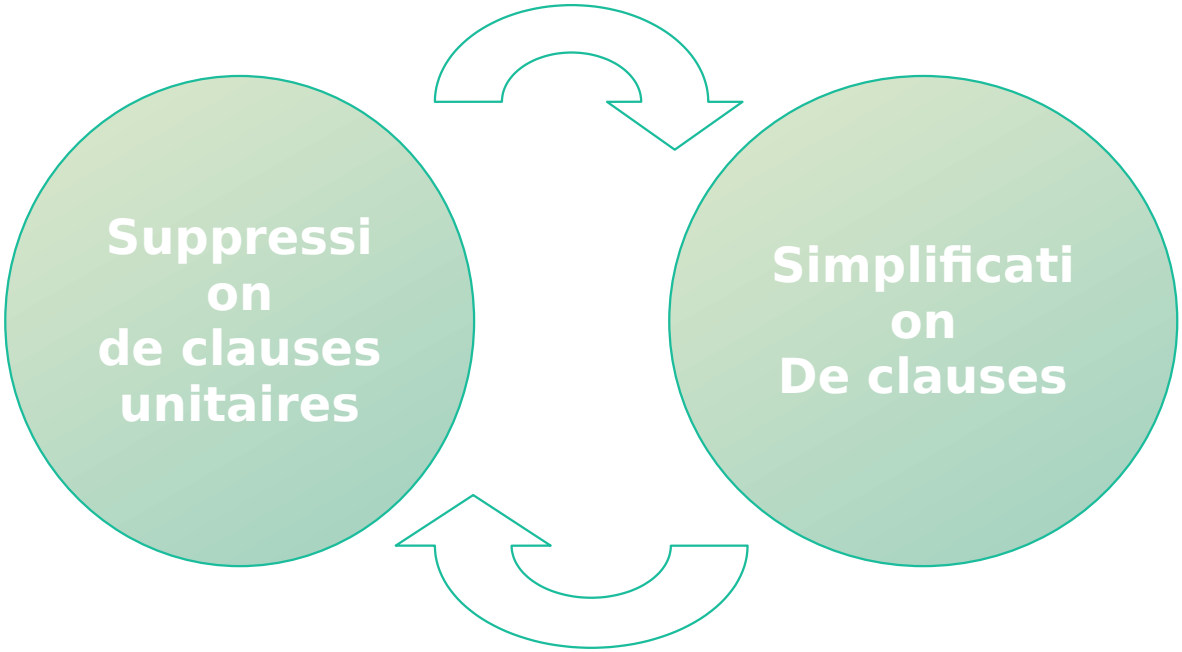
V - Conclusion

II – b) Algorithme CDCL

- Apprentissage par clause

- Retour sur trace non chronologique

- Propagation unitaire



I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

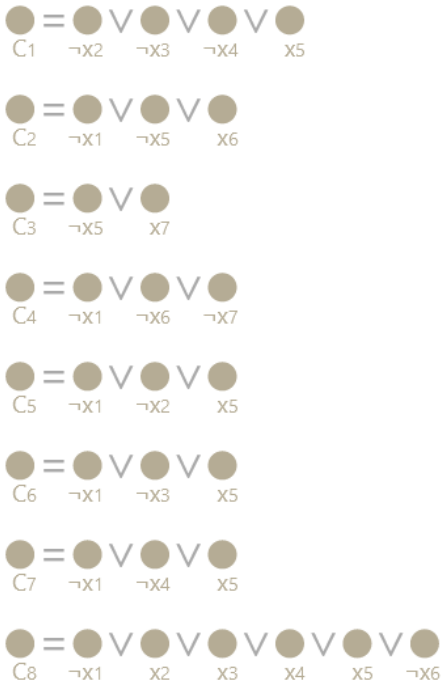
d) Contrainte de règle

e) Résultats

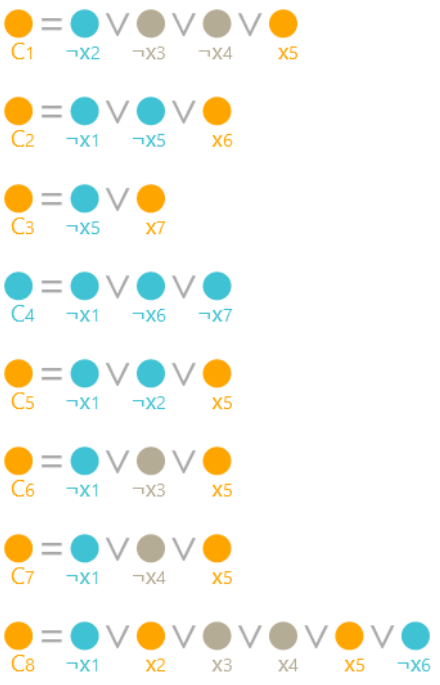
V - Conclusion

II – b) Algorithme CDCL

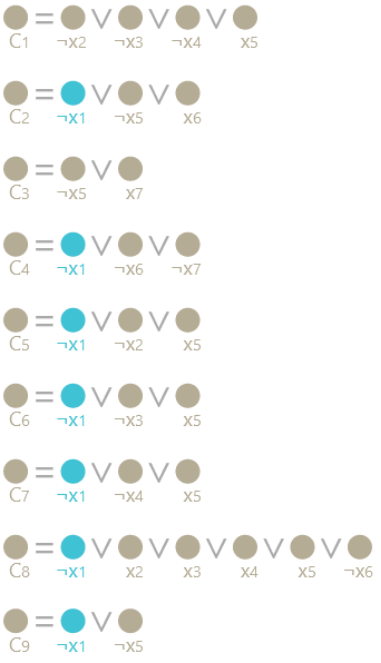
1 -



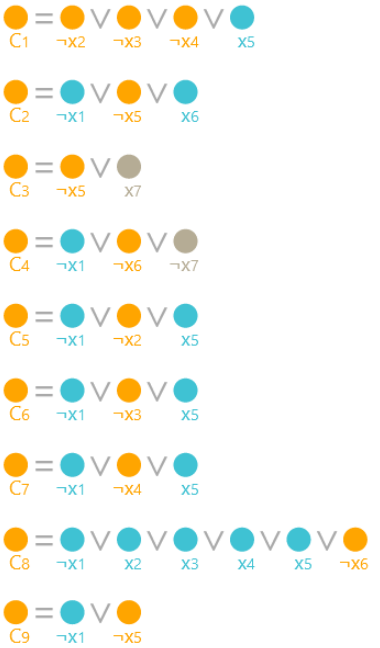
2-



3 -



4 -



I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

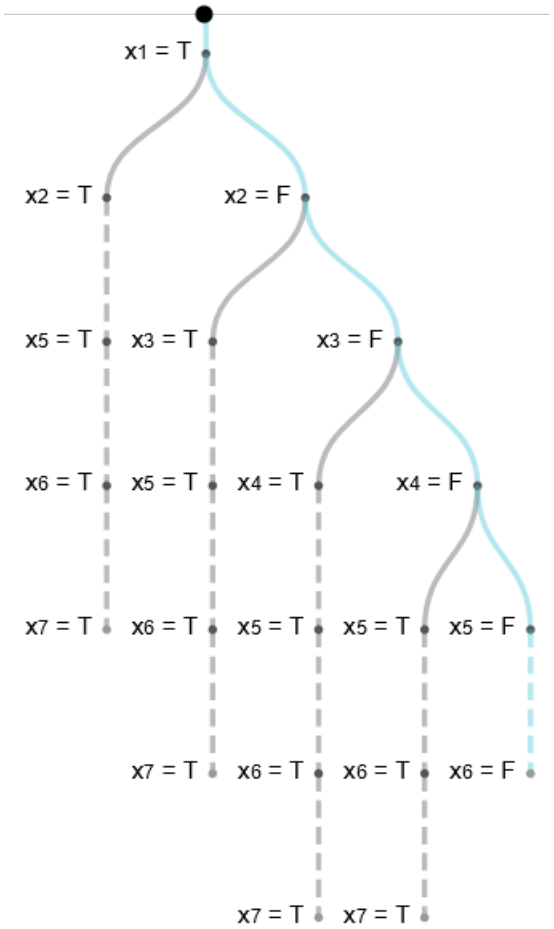
e) Résultats

V - Conclusion

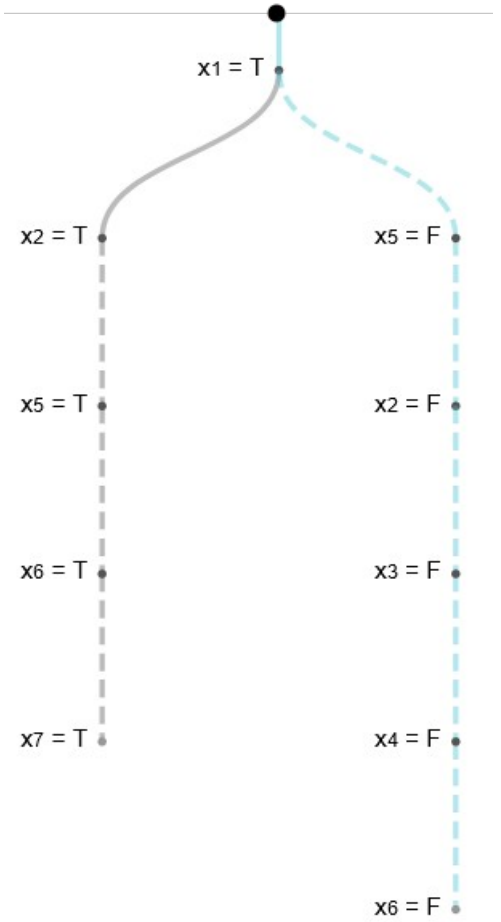
II – b) Algorithme CDCL

Satisfiable
Assignement : { x1 = T ; x2 = F ; x3 = F ; x4 = F ; x5 = F ; x6 = F ; x7 = T/F }

Comparaison DPLL / CDCL :



DPLL



CDCL

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

f) Résultats

V - Conclusion

II – c) Résultats

Échantillon de 1000 grilles aléatoires avec 6 indices :

	Temps min (ms)	Temps max (ms)	Temps moyen (ms)
DPLL	0.073	0.3	0.17
CDCL	0.015	0.21	0.1

Langage :

Python3 (CPython)

Processeur :

- AMD Ryzen 5 5500U
- 2100 MHz
- 6 cœurs
- 12 processeurs logiques

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

e) Résultats

V - Conclusion

III – a) Contrainte Supplémentaire

X SUDOKU : Unicité sur les diagonales

					4			
3		6		1				
7			2		5			8
				9				
				6				
2			8		1			4
				7		8		3
			9					

https://www.clarity-media.com/

Nouvelles clauses :

-Diagonale principale :

$$\bigwedge_{k=1}^{4}\left(\left(\bigvee_{i=1}^{4} x_{i,i}^{k}\right)\wedge\bigwedge_{i,j\in E}\neg x_i^k\vee x_j^k\right)$$

$$E=\{((1,1),(2,2)),((1,1),(3,3)),((1,1),(4,4)),((2,2),(3,3)),((2,2),(4,4)),((3,3),(4,4))\}$$

-Diagonale secondaire :

$$\bigwedge_{k=1}^{4}\left(\left(\bigvee_{i=1}^{4} x_{i,4-i+1}^{k}\right)\wedge\bigwedge_{i,j\in F}\neg x_i^k\vee x_j^k\right)$$

$$F=\{((1,4),(2,3)),((1,4),(3,2)),((1,4),(4,1)),((2,3),(3,2)),((2,3),(4,1)),((3,2),(4,1))\}$$

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

e) Résultats

V - Conclusion

III – b) Preprocessing

- Subsomption

Une clause C1 est dite subsumée par une clause C2 si tous les littéraux de C1 apparaissent dans C2.

Effet → Suppression des clauses subsumées
Exemple :

$$a \wedge (b \vee c) \wedge (\neg a \vee b \vee c) \wedge (a \vee \neg c)$$



$$(\neg a \vee b \vee c) \wedge (a \vee \neg c)$$

- Redondance de clause

Une clause C1 est dite redondante si il existe une clause C2 telle que C1 = C2.

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

e) Résultats

V - Conclusion

III – c) Contrainte de forme

Twodoku :

4	3	1	7			8		5
6			1	8		3		4
	8	7	4		9	1	6	2
8		5		7		9		6
7	6	3	9		8		2	1
1	2	9	6	4	3	7		
	7			1	4		8	
3		8	5	9	2	6	4	7
9		4	8	6	7	2		

		3	2	4				
	9				5			
		4						
1			4	8		7		
9		8	3	2		6	1	
	7	4		1		9	2	8
8	3	2	6		1	5	7	9
4		5	9	7	2			1
7	9	1	8	3		4	6	2

https://www.clarity-media.com/

Samurai Sudoku :

		2			9		8						1			4		8		
		6		5	8		7				1	8		9			6			
	5	1		6	3			9			3		2			1	9			
1	4	3		9	7		5							7		8				
5		7		1				6			5			6	2			9		
		9													9	7		1		
9	1	8					2				1					1				5
7		5		2	1	3	9	8					7	4		2		8		
2	3					7						9			7			6		
									8		6	2		1						
						4	1	6	2		7			9						
								2	9	1	4	7	3							
		3	9			6					9	5	1	2	9				4	
	7				5	1		9				4						7	5	
	9		6		3								9				6	8	2	1
4	2		7				1	6							8			6		5
9		6				4								5			3		1	
	3			8			5						8							
		2	3		1		6	8				1		9			4			
			2									8		3			9		6	2
				4		7		1				6		4		8				3

https://sudoku-puzzles.net/fr/samurai-sudoku/

Résolution

- Résolution des deux grilles
- Ajout de contraintes sur les zones communes

Optimisation

Preprocessing

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

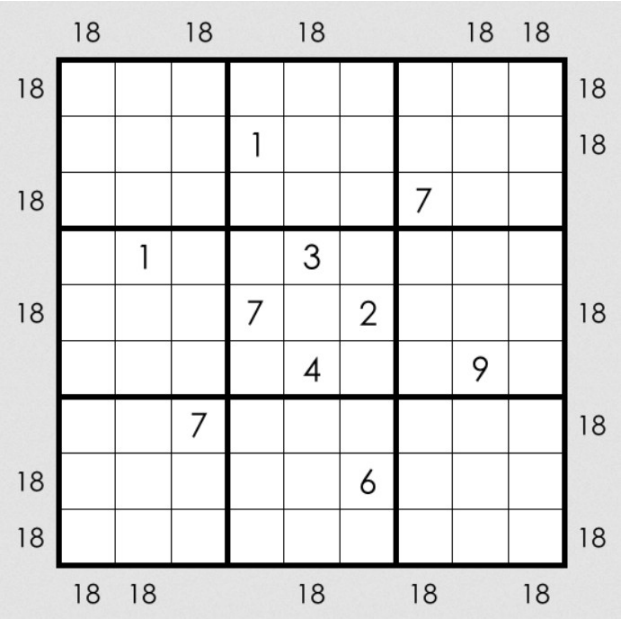
d) Contrainte de règle

e) Résultats

V - Conclusion

III – d) Contrainte de règles

Grille du Championnat de Pologne 9 Mars 2013



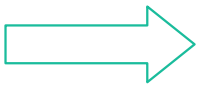
https://sudokuvariante.blogspot.com/

Nouvelles règles :

- Unicité dans les régions

- Contrainte de somme des éléments sur chaque lignes

1	2		4	8
3		3		12
2			3	11
	1	1		9



1	2	1	4	8
3	4	3	2	12
2	4	2	3	11
3	1	1	4	9

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

e) Résultats

V - Conclusion

III – e) Résultats

Pour un échantillon de 1000 grilles aléatoires pré-remplies avec 6 indices :

Variante 1 : Temps de résolution Avec/Sans preprocessing

	Temps min (ms)	Temps max (ms)	Temps moyen (ms)
Sans	0.12	0.5	0.32
Avec	0.08	0.401	0.2

Variante 2 : Temps de résolution Avec/Sans preprocessing

	Temps min (ms)	Temps max (ms)	Temps moyen (ms)
Sans	0.4	0.702	0.56
Avec	0.287	0.51	0.37

Variante 3 : Temps de résolution Avec/Sans heuristique

	Temps min (ms)	Temps max (ms)	Temps moyen (ms)
Sans	0.11	0.402	0.271
Avec	0.002	0.073	0.03

I – Logique Propositionnelle

a) Problème et codage SAT

b) Problème SAT

II - SAT Solver

a) DPLL

b) CDCL

c) Résultats

III - Contraintes

a) Contrainte supplémentaire

b) Preprocessing

c) Contrainte de forme

d) Contrainte de règle

e) Résultats

V - Conclusion

Conclusion

Principales optimisations décelées :

- Preprocessing
- Heuristique

Critique des choix opérés :

- Simplification en 4x4
- Méthodes de preprocessing
- Choix des contraintes

Améliorations :

- Ajouter des éléments de comparaison

Merci pour votre attention

ANNEXE 1 – Encodage des clauses

```python
def variable(row, col, num, board_size):
    return row * board_size**2 + col * board_size + num

def encode_sudoku(board_size, clues):
    clauses = []

    # Contraintes classiques du Sudoku
    for row in range(board_size):
        for col in range(board_size):
            clauses.append([variable(row, col, num, board_size) for num in range(1, board_size + 1)])

    # Contraintes de ligne, colonne et sous-grille
    for num in range(1, board_size + 1):
        for row in range(board_size):
            for col1 in range(board_size):
                for col2 in range(col1 + 1, board_size):
                    clauses.append([-variable(row, col1, num, board_size), -variable(row, col2, num, board_size)])
            for col in range(board_size):
                for row1 in range(board_size):
                    for row2 in range(row1 + 1, board_size):
                        clauses.append([-variable(row1, col, num, board_size), -variable(row2, col, num, board_size)])

        for row_block in range(0, board_size, 2):
            for col_block in range(0, board_size, 2):
                for i in range(2):
                    for j in range(2):
                        for k in range(i * 2 + j + 1, 4):
                            clauses.append([
                                -variable(row_block + i // 2, col_block + i % 2, num, board_size),
                                -variable(row_block + k // 2, col_block + k % 2, num, board_size)
                            ])

    # Les indices des indices pré-remplis sont fixés
    for row in range(board_size):
        for col in range(board_size):
            if clues[row][col] > 0:
                clauses.append([variable(row, col, clues[row][col], board_size)])

    return clauses
```

ANNEXE 2 – Algorithme DPLL

```python
def dpll(clauses, assignment):
    if all_clause_satisfied(clauses, assignment):
        return True, assignment

    if any_clause_unsatisfied(clauses, assignment):
        return False, None

    var = select_unassigned_variable(clauses, assignment)

    for value in [True, False]:
        local_assignment = assignment.copy()
        local_assignment[var] = value
        sat, final_assignment = dpll(clauses, local_assignment)
        if sat:
            return True, final_assignment

    return False, None

def all_clause_satisfied(clauses, assignment):
    for clause in clauses:
        if not any((literal > 0 and assignment.get(abs(literal), False) == True) or
                   (literal < 0 and assignment.get(abs(literal), True) == False) for literal in clause):
            return False
    return True

def any_clause_unsatisfied(clauses, assignment):
    for clause in clauses:
        if all((literal > 0 and assignment.get(abs(literal), None) == False) or
               (literal < 0 and assignment.get(abs(literal), None) == True) for literal in clause):
            return True
    return False

def select_unassigned_variable(clauses, assignment):
    for clause in clauses:
        for literal in clause:
            if abs(literal) not in assignment:
                return abs(literal)
    raise ValueError("No unassigned variables found")
```

ANNEXE 3) a – Algorithme CDCL

```python
def cdcl(clauses):
    assignment = {}
    decision_level = 0
    trail = []
    learnt_clauses = []
    while True:
        conflict_clause = propagate(clauses, assignment, trail, decision_level)
        if conflict_clause:
            if decision_level == 0:
                return False, assignment
            conflict_clause, backjump_level = analyze_conflict(conflict_clause, trail, decision_level)
            learnt_clauses.append(conflict_clause)
            clauses.append(conflict_clause)
            backtrack(trail, assignment, backjump_level)
            decision_level = backjump_level
        else:
            if all_variables_assigned(clauses, assignment):
                return True, assignment
            decision_level += 1
            variable = select_unassigned_variable(clauses, assignment)
            trail.append((variable, True, decision_level))
            assignment[variable] = True

def propagate(clauses, assignment, trail, decision_level):
    for clause in clauses:
        if is_unit_clause(clause, assignment):
            literal = get_unassigned_literal(clause, assignment)
            trail.append((abs(literal), literal > 0, decision_level))
            assignment[abs(literal)] = literal > 0
            if is_conflicting(clause, assignment):
                return clause
    return None
```

ANNEXE 3) b – Algorithme CDCL

```python
def is_unit_clause(clause, assignment):
    unassigned_literals = [literal for literal in clause if abs(literal) not in assignment]
    assigned_true = any(literal > 0 and assignment.get(abs(literal), False) or literal < 0 and not assignment.get(abs(literal), True) for literal in clause)
    return len(unassigned_literals) == 1 and not assigned_true

def get_unassigned_literal(clause, assignment):
    for literal in clause:
        if abs(literal) not in assignment:
            return literal
    raise ValueError("No unassigned literals in unit clause")

def is_conflicting(clause, assignment):
    return all(literal > 0 and assignment.get(abs(literal), None) == False or literal < 0 and assignment.get(abs(literal), None) == True for literal in clause)

def analyze_conflict(conflict_clause, trail, decision_level):
    learnt_clause = conflict_clause[:]
    for variable, value, level in reversed(trail):
        if level < decision_level:
            continue
        if any(literal == variable for literal in conflict_clause):
            learnt_clause.append(-variable if value else variable)
    backjump_level = 0
    return learnt_clause, backjump_level

def backtrack(trail, assignment, backjump_level):
    while trail and trail[-1][2] > backjump_level:
        variable, value, level = trail.pop()
        del assignment[variable]

def all_variables_assigned(clauses, assignment):
    for clause in clauses:
        for literal in clause:
            if abs(literal) not in assignment:
                return False
    return True

def select_unassigned_variable(clauses, assignment):
    for clause in clauses:
        for literal in clause:
            if abs(literal) not in assignment:
                return abs(literal)
    raise ValueError("No unassigned variables found")
```

ANNEXE 4) Fonctions de preprocessing

```python
def subsumption(clauses):
    new_clauses = []
    for i, clause in enumerate(clauses):
        subsumed = False
        for j, other_clause in enumerate(clauses):
            if i != j and set(clause).issubset(set(other_clause)):
                subsumed = True
                break
        if not subsumed:
            new_clauses.append(clause)
    return new clauses
```

```python
def remove_clauses_redondates(clauses):
    unique_clauses = []
    seen = set()
    for clause in clauses:
        clause_tuple = tuple(sorted(clause))
        if clause_tuple not in seen:
            unique_clauses.append(clause)
            seen.add(clause_tuple)
    return unique_clauses
```

ANNEXE 5) Heuristique nouvelle règle

```python
from itertools import combinations
def variable(row, col, num, board_size):
    return row * board_size**2 + col * board_size + num

def encode_sudoku_with_sum(board_size, clues, somme_ligne):
    clauses = []

    # Contraintes classiques du Sudoku
    for row in range(board_size):
        for col in range(board_size):
            clauses.append([variable(row, col, num, board_size) for num in range(1, board_size + 1)])

    # Contraintes de bloc 2x2
    for num in range(1, board_size + 1):
        for row_block in range(0, board_size, 2):
            for col_block in range(0, board_size, 2):
                for i in range(2):
                    for j in range(2):
                        for k in range(i * 2 + j + 1, 4):
                            clauses.append([
                                -variable(row_block + i // 2, col_block + i % 2, num, board_size),
                                -variable(row_block + k // 2, col_block + k % 2, num, board_size)
                            ])

    # Contraintes de somme pour chaque ligne
    for row in range(board_size):
        for combo in combinations(range(1, board_size + 1), board_size):
            if sum(combo) == somme_ligne:
                clauses.append([variable(row, col, num, board_size) for col, num in enumerate(combo)])

    # Les indices des indices pré-remplis sont fixés
    for row in range(board_size):
        for col in range(board_size):
            if clues[row][col] > 0:
                clauses.append([variable(row, col, clues[row][col], board_size)])

    return clauses
```

```python
def choisir_variable_sch_cdcl(assignment, grille, somme_ligne):
    max_diff = -1
    choix = None
    for ligne in range(4):
        somme_actuelle = sum(grille[ligne])
        diff = abs(somme_ligne - somme_actuelle)
        if diff > max_diff:
            for colonne in range(4):
                var = variable(ligne, colonne, 1, 4)
                if grille[ligne][colonne] == 0 and var not in assignment:
                    max_diff = diff
                    choix = var
    return choix
```